

Lab 10: Threads and Locks

This report explains common threading problems in Operating Systems using simple examples. The experiments were tested using Helgrind to detect synchronization issues.

Part 1: Race Condition Example

The first program contains a shared variable called *balance*. Both the main thread and worker thread update this variable at the same time without protection. Because of this, the final result may become incorrect.

Helgrind was executed to check the program behavior. The tool detected a race condition and showed that two threads accessed the same memory location without synchronization.

```
rawan@ubuntu: ~/ostep-homework/threads-api
GNU nano 6.2 main-deadlock.c
#include <stdio.h>
#include "common_threads.h"
pthread_mutex_t m1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t m2 = PTHREAD_MUTEX_INITIALIZER;

void* worker(void* arg) {
    if ((long long) arg == 0) {
        Pthread_mutex_lock(&m1);
        Pthread_mutex_lock(&m2);
    } else {
        Pthread_mutex_lock(&m2);
        Pthread_mutex_lock(&m1);
    }
    Pthread_mutex_unlock(&m1);
    Pthread_mutex_unlock(&m2);
    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t p1, p2;
    Pthread_create(&p1, NULL, worker, (void *) (long long) 0);
    Pthread_create(&p2, NULL, worker, (void *) (long long) 1);
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    return 0;
}

rawan@ubuntu:~/ostep-homework/threads-api$ nano main-deadlock.c
rawan@ubuntu:~/ostep-homework/threads-api$ gcc -o main-deadlock main-deadlock.c -pthread
rawan@ubuntu:~/ostep-homework/threads-api$ valgrind --tool=helgrind ./main-deadlock
==99698== Helgrind, a thread error detector
==99698== Copyright (C) 2007-2017, and GNU GPL'd, by OpenWorks LLP et al.
==99698== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==99698== Command: ./main-deadlock
==99698==
==99698== ---Thread-Announcement-----
==99698== Thread #3 was created
==99698==   at 0x4999A73: clone (clone.S:76)
==99698==   by 0x4999A6E: __clone_internal (clone-internal.c:83)
==99698==   by 0x49086D8: create_thread (pthread_create.c:295)
==99698==   by 0x49091FF: pthread_create@@GLIBC_2.34 (pthread_create.c:828)
==99698==   by 0x4853767: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==99698==   by 0x1093F4: main (in /home/rawan/ostep-homework/threads-api/main-deadlock)
==99698==
==99698== -----
==99698== Thread #3: lock order "0x10C040 before 0x10C080" violated
==99698==
==99698== Observed (incorrect) order is: acquisition of lock at 0x10C080
==99698==   at 0x4850CCF: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==99698==   by 0x109288: worker (in /home/rawan/ostep-homework/threads-api/main-deadlock)
==99698==   by 0x485396A: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==99698==   by 0x4908AC2: start_thread (pthread_create.c:442)
==99698==   by 0x4999A83: clone (clone.S:100)
==99698==
==99698== Followed by a later acquisition of lock at 0x10C040
==99698==   at 0x4850CCF: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==99698==   by 0x1092C3: worker (in /home/rawan/ostep-homework/threads-api/main-deadlock)
==99698==   by 0x485396A: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==99698==   by 0x4908AC2: start_thread (pthread_create.c:442)
==99698==   by 0x4999A83: clone (clone.S:100)
==99698==
==99698== Required order was established by acquisition of lock at 0x10C040
==99698==   at 0x4850CCF: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==99698==   by 0x10920E: worker (in /home/rawan/ostep-homework/threads-api/main-deadlock)
==99698==   by 0x485396A: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==99698==   by 0x4908AC2: start_thread (pthread_create.c:442)
==99698==   by 0x4999A83: clone (clone.S:100)
==99698==
==99698== Followed by a later acquisition of lock at 0x10C080
==99698==   at 0x4850CCF: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==99698==   by 0x109288: worker (in /home/rawan/ostep-homework/threads-api/main-deadlock)
==99698==   by 0x485396A: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==99698==   by 0x4908AC2: start_thread (pthread_create.c:442)
==99698==   by 0x4999A83: clone (clone.S:100)
```

```
rawan@ubuntu: ~/ostep-homework/threads-api
GNU nano 6.2 main-deadlock-fixed.c *
void* worker(void* arg) {
    pthread_mutex_lock(&m1);
    pthread_mutex_lock(&m2);
    pthread_mutex_unlock(&m1);
    pthread_mutex_unlock(&m2);
    return NULL;
}
```

Fixing the Problem

A mutex lock was added around the shared variable access. This allows only one thread to update the variable at a time and prevents incorrect results.

Part 2: Deadlock Example

The second program uses two mutexes and two threads. Each thread locks the mutexes in a different order. This situation may cause both threads to wait forever, which is known as a deadlock.

Helgrind reported a lock ordering issue and warned about a possible deadlock between the two threads.

```
rawan@ubuntu: $ cd ostep-homework/threads-api
rawan@ubuntu:~/ostep-homework/threads-api$ ls
common_threads.h main-deadlock.c main-deadlock-global.c main-race.c main-signal.c main-signal-cv.c Makefile README.md
rawan@ubuntu:~/ostep-homework/threads-api$ gcc -o main-race main-race.c -pthread
rawan@ubuntu:~/ostep-homework/threads-api$ ./main-race
rawan@ubuntu:~/ostep-homework/threads-api$ valgrind --tool=helgrind ./main-race
==99461== Helgrind, a thread error detector
==99461== Copyright (C) 2007-2017, and GNU GPL'd, by OpenWorks LLP et al.
==99461== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==99461== Command: ./main-race
==99461==
==99461== ---Thread-Announcement-----
==99461== Thread #1 is the program's root thread
==99461==
==99461== ---Thread-Announcement-----
==99461== Thread #2 was created
==99461==   at 0x4999A73: clone (clone.S:76)
==99461==   by 0x499A96E: __clone_internal (clone-internal.c:83)
==99461==   by 0x49080D8: create_thread (pthread_create.c:295)
==99461==   by 0x49091FF: pthread_create@@GLIBC_2.34 (pthread_create.c:828)
==99461==   by 0x4853767: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==99461==   by 0x109209: main (in /home/rawan/ostep-homework/threads-api/main-race)
==99461==
==99461== Possible data race during read of size 4 at 0x10C014 by thread #1
==99461== Locks held: none
==99461==   at 0x109236: main (in /home/rawan/ostep-homework/threads-api/main-race)
==99461==
==99461== This conflicts with a previous write of size 4 by thread #2
==99461== Locks held: none
==99461==   at 0x1091BE: worker (in /home/rawan/ostep-homework/threads-api/main-race)
==99461==   by 0x485396A: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==99461==   by 0x4908AC2: start_thread (pthread_create.c:442)
==99461==   by 0x4999A83: clone (clone.S:100)
==99461== Address 0x10C014 is 0 bytes inside data symbol "balance"
==99461==
==99461== Possible data race during write of size 4 at 0x10C014 by thread #1
==99461== Locks held: none
==99461==   at 0x10923F: main (in /home/rawan/ostep-homework/threads-api/main-race)
==99461==
```

```
rawan@ubuntu: ~/ostep-homework/threads-api
GNU nano 6.2 main-race-fixed.c *
#include <stdio.h>
#include <pthread.h>

int balance = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void* worker(void* arg) {
    pthread_mutex_lock(&lock);
    balance++;
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t p;
    pthread_create(&p, NULL, worker, NULL);
    pthread_mutex_lock(&lock);
    balance++;
    pthread_mutex_unlock(&lock);
    pthread_join(p, NULL);
    printf("Final balance: %d\n", balance);
    return 0;
}

rawan@ubuntu:~/ostep-homework/threads-api$ gcc -o main-race-fixed main-race-fixed.c -pthread
rawan@ubuntu:~/ostep-homework/threads-api$ valgrind --tool=helgrind ./main-race-fixed
==99588== Helgrind, a thread error detector
==99588== Copyright (C) 2007-2017, and GNU GPL'd, by OpenWorks LLP et al.
==99588== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==99588== Command: ./main-race-fixed
==99588==
Final balance: 2
==99588==
==99588== Use --history-level=approx or =none to gain increased speed, at
==99588== the cost of reduced accuracy of conflicting-access information
==99588== For lists of detected and suppressed errors, rerun with: -s
==99588== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 7 from 7)
```

Deadlock Solution

The issue was solved by forcing both threads to lock the mutexes in the same order every time. This removes the circular waiting problem.

Conclusion

This lab demonstrated how synchronization errors can happen in multithreaded programs. Using mutexes and proper lock ordering helps avoid race conditions and deadlocks.